

Place and Route of a 4-bit Gray Counter

From the Lab 2 netlist to a routed layout in IC Compiler

Bibek Shrestha | October 2026 | Synopsys IC Compiler, SAED 90 nm

The assignment

Take the synthesized gray counter from the previous lab through automatic placement and routing with IC Compiler: floorplan, power plan, placement, clock tree synthesis and routing. Generate and analyze reports.

At a glance

	After CTS	After routing
Clock target	5 ns	5 ns
Critical path	[ns]	[ns]
Setup slack	[ns]	[ns]
Hold violations	[]	[]
Cells (logic + flip-flops)	[]	[]
Cell area	[]	[]
Utilization	[%]	[%]
Net length	[μ m]	[μ m]
Open nets / DRCs		[] / []

Table 1: Results from my QoR, utilization and verify_zrt_route reports.

1 The design

A 4-bit counter that steps through Gray code, so only one bit changes per step: 0, 1, 3, 2, 6, 7, 5, 4, C, D, F, E, A, B, 9, 8, then back to 0. It has an active-low asynchronous reset `reset_n` and an enable `en_count`. The case statement is just a 16-entry next-state table.

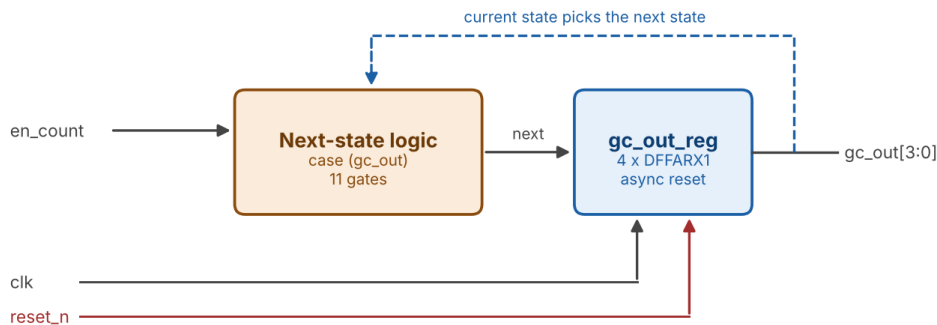


Figure 1: What the RTL becomes: a 4-bit register with the next-state table in front of it.

Synthesis mapped my version to 4 DFFARX1 flip-flops and 11 logic gates (MUX21X1, XOR2X1, NAND2X0, NOR2X0, AND2X1, INVX0). The tutorial's run used A022X1 cells instead of muxes. Same logic, different gate choice.

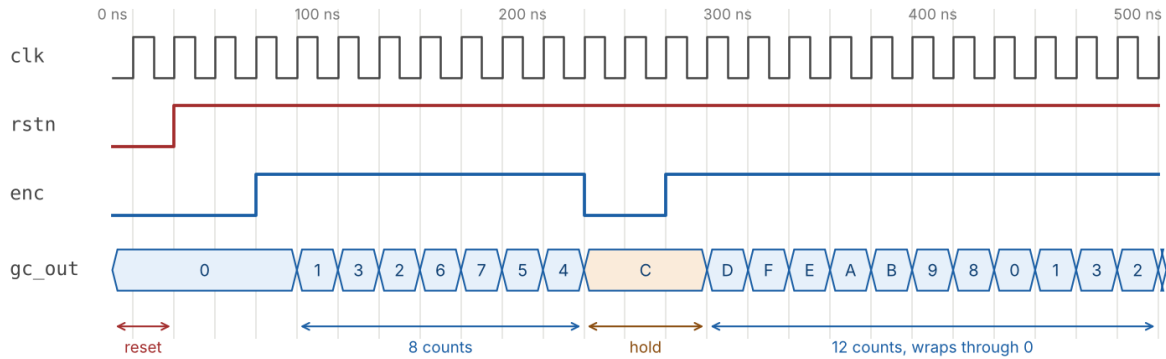


Figure 2: Expected output for the given testbench (20 ns clock). Changes made by the testbench on a clock edge are seen one edge later.

2 Workflow

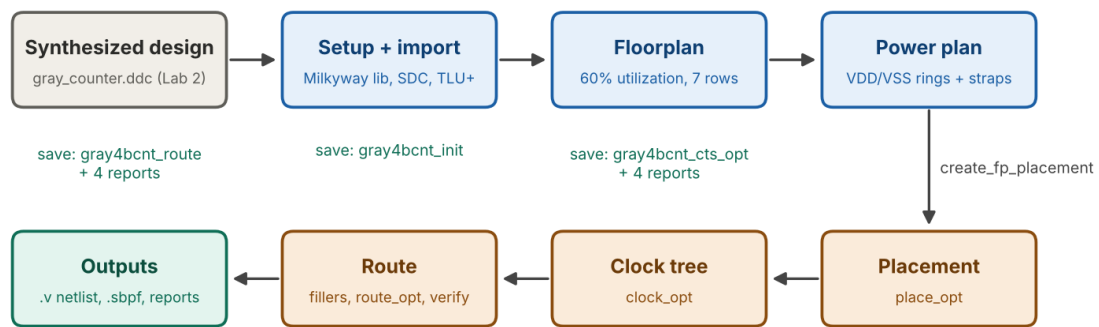


Figure 3: Place and route flow. Each green label is a checkpoint saved in the Milkyway library.

1. **Setup:** point ICC to the SAED90 libraries and create the Milkyway library gray4bcntLIB with the technology file and the cell abstracts (FRAM).
2. **Import** gray_counter.ddc from Lab 2, read the SDC (5 ns clock), set the TLU+ wire models.
3. **Floorplan and power plan**, then a quick create_fp_placement.
4. **Placement and CTS** with place_opt and clock_opt, then the first set of reports.
5. **Fillers and routing** with route_opt, check with verify_zrt_route, second set of reports.
6. **Extract** parasitics and write the final netlist.

```
cd syn_tut/icc                                # one level below output/ and constraints/
icc_shell -shared_license -f ../src/icc_flow.tcl | tee icc_output.txt
```

3 Floorplan and power plan

Setting	Value
Core utilization	0.6 (cells fill 60% of the core, the rest is for wiring)
Core to die gap	30 μm on all four sides
VSS ring / VDD ring	1 μm wide, offset 0.5 / 1.8 μm ; M6 vertical, M7 horizontal
Power straps	VDD and VSS, M6, vertical, 3 μm wide

Table 2: Floorplan settings (same as the tutorial).

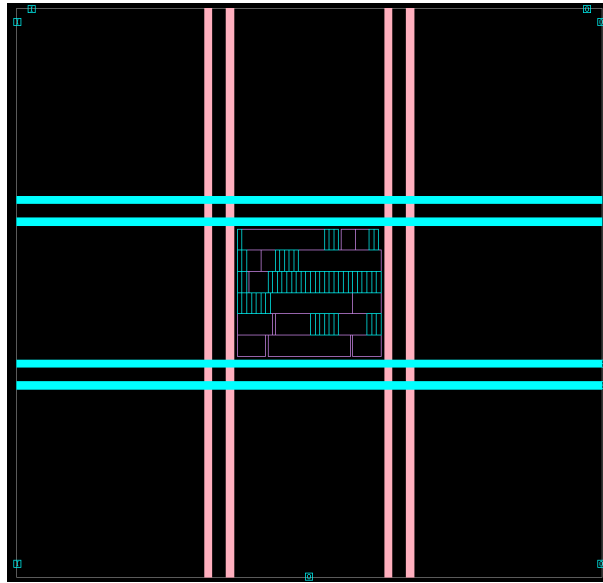


Figure 4: After `create_fp_placement`: rings (cyan), straps (pink) and the cells placed in rows in the middle.

4 Placement, clock tree and routing

`place_opt` legalized the cells and checked timing. `clock_opt` built the clock network to the 4 flip-flops. Fillers (`SHFILL2`) closed the gaps in the rows so the power rails run unbroken, and `route_opt` drew the signal wires.

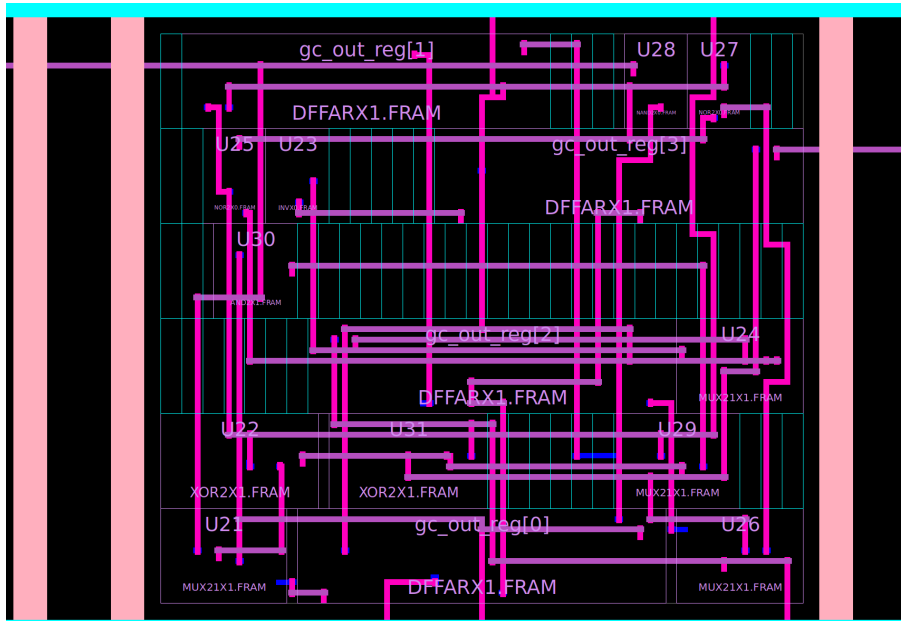


Figure 5: Zoom on the routed block. Labels are my cell instances; pink and violet are routed metal.

5 Reports

Report	What I checked	Result
*_qor.rpt	slack, violating paths	☐
*.setup.rpt	worst setup path	[start → end, slack]
*.hold.rpt	worst hold path	☐
*_util.rpt	standard cell utilization	☐
verify_zrt_route	open nets, DRCs	☐

Table 3: What each report says for my run.

6 Inside the cells

ICC places and routes with the abstract (FRAM) view of each cell: only the outline, the pins and the blocked metal. The full layout (CEL view) has every mask layer.

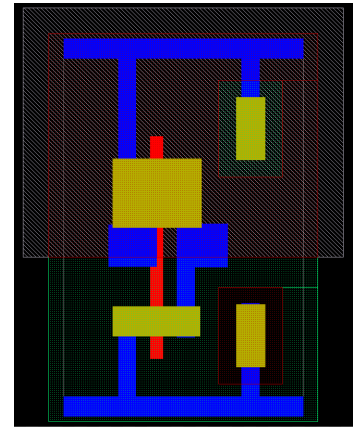
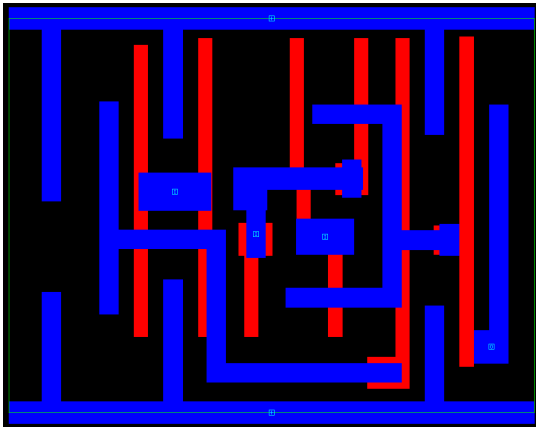


Figure 6: Left: MUX21X1 FRAM view, with VDD/VSS rails at top and bottom and the pin squares in between. Right: INVX0 CEL view. PMOS sits in the N-well (hatched, top), NMOS below, and one poly gate (red) drives both.

7 What I learned

- Physical design starts from the synthesized .ddc, so the gates are already fixed; ICC only decides where they go and how they connect.
- Utilization is a trade-off: a smaller core saves area but leaves less room to route.
- The power grid (rings, straps, cell rails) is built before any cell is placed.
- The PATH in the tutorial did not exist on our server. ICC is installed at

```
/apps/synopsys2023/icc/V-2023.12-SP5/bin
```

- create_mw_lib runs only once. After that, use open_mw_lib.

A Source files

RTL: src/gray_counter.sv

```

/*****
  MODULE: graycount, Sequential Circuit Example: gray_counter.sv
  *****/

module gray_counter (reset_n, clk, en_count, gc_out);
    input logic reset_n;          // active-low RESET signal
    input logic clk;              // clock signal
    input logic en_count;         // counting is enabled when en_count = 1
    output logic [3:0] gc_out;    // current value of counter

    // Compute new gc_out value based on current gc_out value

    always_ff @(negedge reset_n or posedge clk)
    begin
        if (~reset_n)
            gc_out <= 0;
        else begin
            if (en_count) begin
                case (gc_out)
                    4'b0000: gc_out <= 4'b0001;
                    4'b0001: gc_out <= 4'b0011;

```

```

        4'b0011: gc_out <= 4'b0010;
        4'b0010: gc_out <= 4'b0110;
        4'b0110: gc_out <= 4'b0111;
        4'b0111: gc_out <= 4'b0101;
        4'b0101: gc_out <= 4'b0100;
        4'b0100: gc_out <= 4'b1100;
        4'b1100: gc_out <= 4'b1101;
        4'b1101: gc_out <= 4'b1111;
        4'b1111: gc_out <= 4'b1110;
        4'b1110: gc_out <= 4'b1010;
        4'b1010: gc_out <= 4'b1011;
        4'b1011: gc_out <= 4'b1001;
        4'b1001: gc_out <= 4'b1000;
        4'b1000: gc_out <= 4'b0000;
        default: gc_out <= 4'b0000;
    endcase
end // end of if (en_count)
end // end of else
end // end of module
endmodule

```

Testbench: src/gray_counter_tb.sv

```

`timescale 1ns/1ps

module gray_counter_tb;

    // internal nets
    logic clk;
    logic rstn;
    logic enc;
    logic [3:0] out;

    // instantiate gray_counter
    gray_counter gc (
        .reset_n (rstn),
        .clk      (clk),
        .en_count (enc),
        .gc_out   (out));

    // clock
    always #10 clk = ~clk;

    // setup tests
    initial begin
        {clk, rstn, enc} <= 0;

        $monitor ("T=%0t rstn=%0b clk=%0b enc=%0b output=0x%0h", $time, rstn, clk, enc, out);

        repeat(2) @ (posedge clk); // repeat block of statements 2 times
        rstn <= 1;
        repeat(2) @ (posedge clk);
        enc <= 1;
        repeat(8) @ (posedge clk);
        enc <= 0;
        repeat(2) @ (posedge clk);
        enc <= 1;
        repeat(12) @ (posedge clk);

        $finish;
    end
endmodule

```

ICC script: src/icc_flow.tcl

```

gui_start
set_app_var search_path /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/
synopsys/models/
set_app_var target_library "saed90nm_max.db"
set_app_var link_library "* saed90nm_max.db"
create_mw_lib -technology "/apps/designlib/SAED90_EDK/SAED_EDK90nm/Technology_Kit/milkyway/
saed90nm_icc_1p9m.tf" -mw_reference_library "/apps/designlib/SAED90_EDK/SAED_EDK90nm/
Digital_Standard_cell_Library/process/astro/fram/saed90nm/" "gray4bcntLIB"
open_mw_lib gray4bcntLIB
import_designs "../output/gray_counter.ddc" -format ddc -top "gray_counter" -cel "gray_counter"
read_sdc ../constraints/gray_counter.sdc
set_tlu_plus_files -max_tluplus "/apps/designlib/SAED90_EDK/SAED_EDK90nm/Technology_Kit/starrcxt/
tluplus/saed90nm_1p9m_1t_Cmax.tluplus" -min_tluplus "/apps/designlib/SAED90_EDK/SAED_EDK90nm/
Technology_Kit/starrcxt/tluplus/saed90nm_1p9m_1t_Cmin.tluplus" -tech2itf_map "/apps/designlib/
SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/process/astro/tech/tech2itf.map"
save_mw_cel -as gray4bcnt_init
create_floorplan -core_utilization 0.6 -start_first_row -left_io2core "30" -bottom_io2core "30" -
right_io2core "30" -top_io2core "30"
derive_pg_connection -power_net "VDD" -power_pin "VDD" -ground_net "VSS" -ground_pin "VSS" -
create_ports "top"
create_rectangular_ring -nets {VSS} -left_offset 0.5 -left_segment_layer M6 -left_segment_width
1.0 -extend_ll -extend_lh -right_offset 0.5 -right_segment_layer M6 -right_segment_width 1.0 -
extend_rl -extend_rh -bottom_offset 0.5 -bottom_segment_layer M7 -bottom_segment_width 1.0 -
extend_bl -extend_bh -top_offset 0.5 -top_segment_layer M7 -top_segment_width 1.0 -extend_tl -
extend_th
create_rectangular_ring -nets {VDD} -left_offset 1.8 -left_segment_layer M6 -left_segment_width
1.0 -extend_ll -extend_lh -right_offset 1.8 -right_segment_layer M6 -right_segment_width 1.0 -
extend_rl -extend_rh -bottom_offset 1.8 -bottom_segment_layer M7 -bottom_segment_width 1.0 -
extend_bl -extend_bh -top_offset 1.8 -top_segment_layer M7 -top_segment_width 1.0 -extend_tl -
extend_th
create_power_strap -nets {VSS} -layer M6 -direction vertical -width 3
create_power_strap -nets {VDD} -layer M6 -direction vertical -width 3
create_fp_placement
place_opt
clock_opt
save_mw_cel -as gray4bcnt_cts_opt
report_placement_utilization > ../output/gray4bcnt_cts_util.rpt
report_qor > ../output/gray4bcnt_cts_qor.rpt
report_timing -delay max -max_paths 5 > ../output/gray4bcnt_cts.setup.rpt
report_timing -delay min -max_paths 5 > ../output/gray4bcnt_cts.hold.rpt
insert_stdcell_filler -cell_with_metal "SHFILL2" -cell_without_metal "SHFILL2" -connect_to_power {
VDD} -connect_to_ground {VSS}
route_opt
verify_zrt_route
report_placement_utilization > ../output/gray4bcnt_route_util.rpt
report_qor > ../output/gray4bcnt_route_qor.rpt
report_timing -delay max -max_paths 5 > ../output/gray4bcnt_route.setup.rpt
report_timing -delay min -max_paths 5 > ../output/gray4bcnt_route.hold.rpt
extract_rc -coupling_cap
write_parasitics -format SBPF -output ../output/gray4bcnt.output.sbpf
write_verilog ../output/gray4bcntLIB.output.v
save_mw_cel -as gray4bcnt_route
close_mw_cel

```